IoT Project Proposal — Part 2

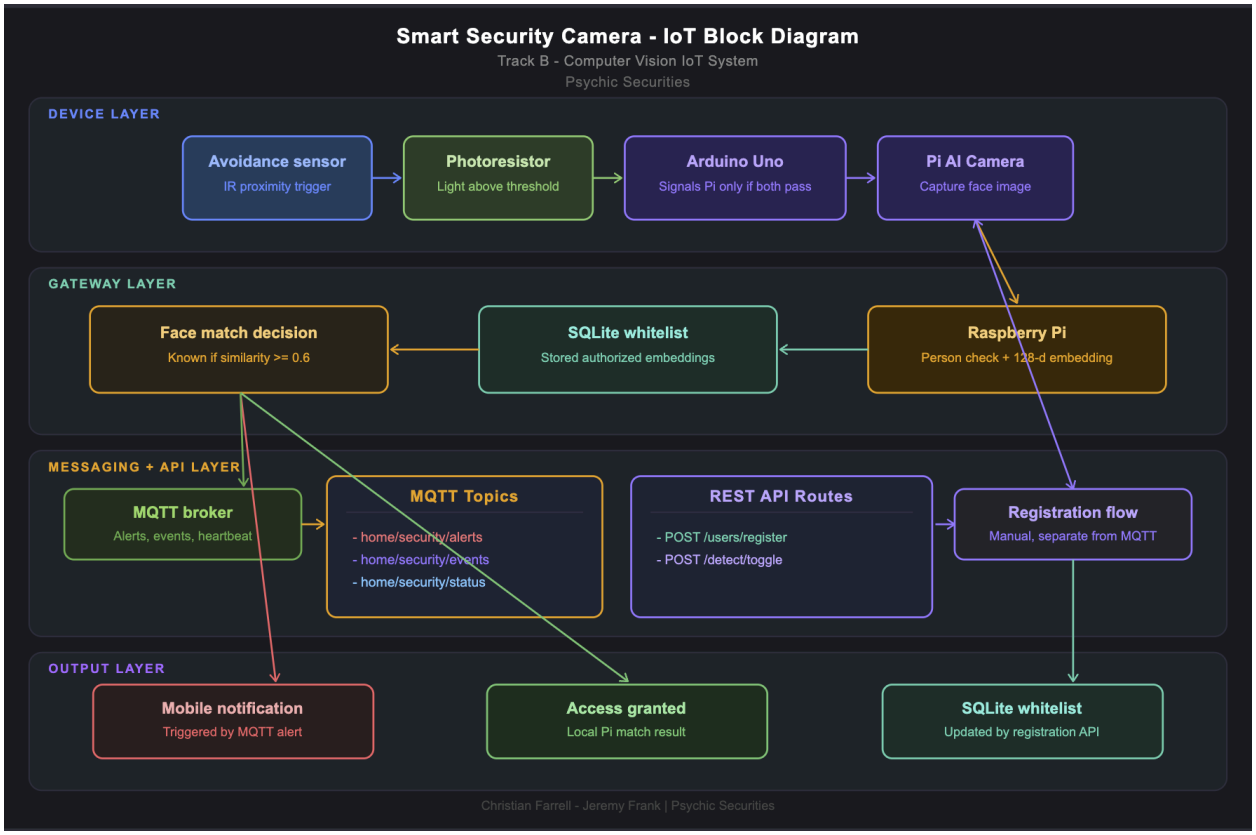
Smart Security Camera for Verified User Access

Track B — Computer Vision IoT System

Christian Farrell & Jeremy Frank

1. Architecture Diagram

The diagram below shows the full detection flow from device input through the gateway to output.



Detection flow: The avoidance sensor detects a nearby object (person approaching the door) and signals the Arduino. The Arduino reads the photoresistor to verify sufficient lighting conditions. If an object is detected and the lighting is above the threshold, the Arduino signals the Raspberry Pi to capture a face image. The image is sent to the Raspberry Pi gateway, which runs the *face_recognition* Python library to generate a 128-dimensional face embedding. This embedding is compared against the stored whitelist in a local SQLite database.

If the embedding matches a known user (cosine similarity above 0.6 threshold), the system grants access. If no match is found, the system publishes an alert to the MQTT broker, which triggers a mobile notification to the homeowner.

This dual-sensor approach reduces false triggers by requiring both proximity (avoidance sensor) and valid lighting (photoresistor) before activating the camera.

Registration flow (separate): A user sends a POST request to the REST API running on the Pi. The API triggers the camera to capture a photo, generates a face embedding, and stores it in the SQLite whitelist. This flow is initiated manually and does not involve MQTT.

2. Topic Plan

The system uses three MQTT topics for real-time event messaging. The REST API handles private operations (registration, detection toggle) and is not exposed via MQTT.

MQTT Topic	Purpose
home/security/alerts	Published when the system detects an unknown face. Contains timestamp, confidence score, and image reference. This topic drives mobile notifications.
home/security/events	General event stream. Publishes when proximity is detected, access is granted, or detection is toggled on/off. Used for logging and monitoring.
home/security/status	Periodic heartbeat is published every 60 seconds. Contains uptime and current detection state (enabled/disabled). Used to confirm the system is running.

3. Event Schema

Below is an example JSON event message published to **home/security/alerts** when an unknown face is detected:

```
{
  "topic": "home/security/alerts",
  "timestamp": "2026-04-14T18:32:05Z",
  "event_type": "unknown_face_detected",
  "confidence": 0.34,
  "image_ref": "/captures/2026-04-14_183205.jpg",
  "sensor_id": "front_door_cam"
}
```

A **low confidence** value means the captured face did not closely match any whitelist entry. If the confidence score is below the 0.6 threshold, the system treats the person as unknown and publishes this alert.

4. Hardware List

Component	Role
Raspberry Pi 4	IoT gateway. Runs the face_recognition library, camera object classification model, Flask REST API, and MQTT client.
Pi AI Camera Module	Captures videos and images when triggered by the motion sensor or REST API, and runs an object detection model to find people in the image taken.
Photoresistor Sensor	Detects ambient light level (used for day/night filtering, not motion detection).
Avoidance Sensor	Avoidance Sensor - Detects nearby objects (e.g., person approaching door) using infrared proximity sensing.
Arduino Uno	Reads avoidance sensor (digital proximity detection) and photoresistor (analog light level). Signals the Raspberry Pi to begin image capture only when both proximity and lighting conditions meet defined thresholds.
Breadboard + jumper wires	Breadboard + jumper wires - Physical wiring between sensors (photoresistor and avoidance sensor), Arduino, and Raspberry Pi
MicroSD card (64 GB)	Stores the OS, application code, and SQLite whitelist database.
LED / buzzer (optional)	Local visual or audio feedback when access is granted or denied.

5. Team Roles

Member	Primary Responsibility
Christian Farrell	MQTT/REST Configuration and Sensor Activation
Jeremy Frank	Camera Accuracy and Embedding Generation

Both members contribute to integration testing and documentation.

6. Reliability / Operations Note

Likely failure modes: The camera may capture a blurry or poorly lit image, leading to unreliable face-embedding comparisons (false rejects or false accepts). Additionally, the avoidance sensor may trigger on non-human objects (e.g., animals or passing objects), while the photoresistor may misinterpret sudden lighting changes such as shadows or headlights, causing false activations.

Detection and mitigation: Before performing face matching, the system evaluates image quality using OpenCV's Laplacian variance (for blur) and a brightness histogram check. If quality is below threshold, the system skips matching, logs a "low quality capture" event to home/security/events, and retries on the next trigger. To reduce false activations, the system only initiates image capture when both proximity (avoidance sensor HIGH within ~10–30 cm) and sufficient lighting (photoresistor value > 300) conditions are met, and the camera's object detection model confirms the presence of a person before running face recognition.

7. Security / Safety Note

Realistic risk: If the REST API is exposed without authentication, anyone on the local network could register themselves as a trusted user via POST /users/register, bypassing the security system entirely.

Safeguard: The REST API requires a pre-shared API key passed in the request header. Additionally, the Flask server binds only to the local network interface (not 0.0.0.0), so it is not reachable from the public internet. Registration requests without a valid key are rejected with a 401 Unauthorized response.

8. Build Plan

By the next checkpoint, we expect to have the following working:

- Avoidance sensor and photoresistor wired to Arduino, triggering the camera capture on the Raspberry Pi only when both proximity and lighting thresholds are met and triggering the camera capture on the Raspberry Pi.
- *face_recognition* library installed and working — able to generate and store a face embedding in SQLite via the REST API.
- Basic face comparison function: capture a face, compare against the whitelist, and output "known" or "unknown" to the console.
- MQTT broker (Mosquitto) running locally and receiving test alert messages.

This keeps the core detection loop end-to-end testable before we layer on mobile notifications or polish the API.